# Description Logic, OWL, and Semantic Reasoning

**A Brief Introduction**

October 9, 2026

**Abstract**

These notes introduce *description logics* (DLs), the family of decidable fragments of first-order logic that underpins the Web Ontology Language (OWL) and, more broadly, automated reasoning over structured knowledge. We give the syntax and model-theoretic semantics, decode the compositional naming scheme ($\mathcal{ALC}$, $\mathcal{S}$, $\mathcal{SHIN}$, $\mathcal{SHOIQ}$, $\mathcal{SROIQ}$), state the standard reasoning tasks and their mutual reducibility, sketch the tableau method that decides them, and summarise the complexity landscape that motivates the OWL 2 profiles.

## 1 What problem are we solving?

Suppose we want a machine to *draw conclusions* from a body of knowledge — not merely to store and retrieve facts, but to infer facts never explicitly written down. If we assert that every Mother is a Woman with at least one child, and that Alice is a Mother, the machine should conclude Alice is a Woman and that she has a child, *even if no such child is named.*

Full first-order logic can express all of this, but its entailment problem is *undecidable*: no algorithm is guaranteed to halt with the right answer. Description logics are a deliberate retreat. They carve out fragments expressive enough for real ontologies — taxonomies in biology and medicine, product catalogues, the Semantic Web — while keeping entailment *decidable*, and ideally practical. The art of the field is choosing *which* language features to admit, because each one costs computationally.

## 2 Syntax: concepts, roles, individuals

A DL vocabulary (a *signature*) has three kinds of name:

- **concept names** (unary predicates): Woman, Mother, Disease — these denote *sets*;
- **role names** (binary predicates): hasChild, partOf — these denote *relations*;
- **individual names** (constants): Alice, Bob.

From atomic concepts we build *complex* concepts with constructors. The baseline language $\mathcal{ALC}$ (*Attributive Language with Complement*) offers:

$$C, D \;::=\; A \mid \top \mid \bot \mid \neg C \mid C \sqcap D \mid C \sqcup D \mid \exists R.C \mid \forall R.C,$$

where $A$ is a concept name and $R$ a role. Read $\exists R.C$ as "has some $R$-filler lying in $C$" and $\forall R.C$ as "all $R$-fillers lie in $C$".

An ontology is split into two parts:

- the **TBox** (terminology): general axioms $C \sqsubseteq D$ ("$C$ is subsumed by $D$") and definitions $C \equiv D$;
- the **ABox** (assertions): facts $C(a)$ and $R(a, b)$ about named individuals.

**Example 1.** The TBox axiom

$$\mathsf{Mother} \;\equiv\; \mathsf{Woman} \sqcap \exists\, \mathsf{hasChild}.\mathsf{Person}$$

*defines* a mother as a woman with at least one child who is a person. With the ABox fact $\mathsf{Mother}(\mathsf{Alice})$, a reasoner derives $\mathsf{Woman}(\mathsf{Alice})$.

## 3 Semantics: everything is sets and relations

The meaning of a DL is Tarskian and strikingly simple. An **interpretation** $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ consists of a non-empty *domain* $\Delta^{\mathcal{I}}$ together with an interpretation function $\cdot^{\mathcal{I}}$ that maps

$$\text{concept } C \mapsto C^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}, \qquad \text{role } R \mapsto R^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}, \qquad \text{individual } a \mapsto a^{\mathcal{I}} \in \Delta^{\mathcal{I}}.$$

The constructors are interpreted exactly as their names suggest:

$$\top^{\mathcal{I}} = \Delta^{\mathcal{I}}, \quad \bot^{\mathcal{I}} = \varnothing, \quad (\neg C)^{\mathcal{I}} = \Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}, \quad (C \sqcap D)^{\mathcal{I}} = C^{\mathcal{I}} \cap D^{\mathcal{I}},$$

$$(\exists R.C)^{\mathcal{I}} = \{\, x \mid \exists y.\, (x,y) \in R^{\mathcal{I}} \;\wedge\; y \in C^{\mathcal{I}} \,\}, \qquad (\forall R.C)^{\mathcal{I}} = \{\, x \mid \forall y.\, (x,y) \in R^{\mathcal{I}} \;\rightarrow\; y \in C^{\mathcal{I}} \,\}.$$

An interpretation $\mathcal{I}$ *satisfies* an axiom $C \sqsubseteq D$ iff $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$, and satisfies $C(a)$ iff $a^{\mathcal{I}} \in C^{\mathcal{I}}$. A **model** of an ontology is an interpretation satisfying *all* its axioms and assertions. Reasoning means asking what holds in *every* model.

Two semantic conventions are worth flagging, because they surprise newcomers:

- **Open-world assumption (OWA).** What is not entailed is *unknown*, not false. If the ABox does not say Alice has a daughter, the reasoner does not conclude she has none.
- **No unique-name assumption (by default).** Two individual names may denote the same domain element unless an axiom forbids it. This interacts sharply with number restrictions (below).

## 4 The naming scheme is a bill of materials

DL names are not arbitrary labels: each letter is a feature flag, and the name lists exactly which constructors the logic admits. The usual base is $\mathcal{S}$, itself shorthand for

$$\mathcal{S} \;=\; \mathcal{ALC} + \text{transitive roles } (R^{+}),$$

so named because transitive roles ($\mathsf{partOf}$, $\mathsf{ancestorOf}$) are ubiquitous. Additional letters extend $\mathcal{S}$:

| Letter | Feature | OWL vocabulary |
|---|---|---|
| $\mathcal{H}$ | role **H**ierarchies ($R \sqsubseteq S$) | `rdfs:subPropertyOf` |
| $\mathcal{O}$ | nominals: singleton / enumerated classes $\{a\}$ | `owl:oneOf`, `owl:hasValue` |
| $\mathcal{I}$ | **I**nverse roles ($R^{-}$) | `owl:inverseOf` |
| $\mathcal{N}$ | unqualified **N**umber restrictions ($\geq n\, R$) | `owl:cardinality` |
| $\mathcal{Q}$ | **Q**ualified number restrictions ($\geq n\, R.C$) | `owl:qualifiedCardinality` |
| $\mathcal{F}$ | **F**unctional roles | `owl:FunctionalProperty` |
| $\mathcal{R}$ | complex **R**ole inclusions (chains), (ir)reflexivity | `owl:propertyChainAxiom` |
| $(\mathcal{D})$ | concrete **D**atatypes (integers, strings, …) | datatype properties |

The logics named in these notes now decode on sight:

- $\mathcal{SHIN} = \mathcal{S} + \mathcal{H} + \mathcal{I} + \mathcal{N}$: hierarchies, inverse roles, unqualified cardinalities — but *no* nominals.

- $\mathcal{SHOIQ} = \mathcal{S} + \mathcal{H} + \mathcal{O} + \mathcal{I} + \mathcal{Q}$: adds nominals and upgrades to *qualified* cardinalities.
- $\mathcal{SHOIN}(\mathcal{D})$ is the logic behind **OWL 1 DL**; it differs from $\mathcal{SHOIQ}$ only in using unqualified $\mathcal{N}$ instead of $\mathcal{Q}$.
- $\mathcal{SROIQ}(\mathcal{D})$ is the logic behind **OWL 2 DL**: $\mathcal{SHOIQ}$ with $\mathcal{H}$ generalised to the richer role box $\mathcal{R}$.

The step from $\mathcal{N}$ to $\mathcal{Q}$ looks cosmetic but is not: $\geq 2\,\mathsf{hasChild}$ merely counts children, whereas $\geq 2\,\mathsf{hasChild}.\mathsf{Female}$ counts children *of a given kind*. Since $\geq n\,R$ is $\geq n\,R.\top$, qualified restrictions strictly subsume unqualified ones — which is why $\mathcal{N}$ and $\mathcal{Q}$ never appear together.

## 5 Reasoning tasks — and why one engine suffices

Over a given ontology, the standard decision problems are:

**Consistency**
: Does the ontology have any model at all?

**Concept satisfiability**
: Is there a model in which $C^{\mathcal{I}} \neq \varnothing$?

**Subsumption**
: Is $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$ in *every* model (written $C \sqsubseteq D$)?

**Instance checking**
: Does $a^{\mathcal{I}} \in C^{\mathcal{I}}$ hold in every model?

**Classification / realization**
: Compute the full subsumption lattice of named concepts / the most specific types of each individual.

The organising fact of the field is that these all **reduce to (un)satisfiability**, provided the logic has negation:

$$C \sqsubseteq D \iff C \sqcap \neg D \text{ is unsatisfiable}, \qquad a : C \iff \mathrm{ABox} \cup \{\neg C(a)\} \text{ is inconsistent.}$$

A reasoner therefore needs only *one* core engine — a satisfiability test — and everything else is a wrapper around it.

## 6 How the engine works: tableau

The dominant decision procedure is the **tableau algorithm**. To test whether $C$ is satisfiable, it attempts to *build a model*: a graph of labelled nodes (individuals) and labelled edges (role fillers), grown by repeatedly applying expansion rules to the node labels $L(x)$:

- $\sqcap$-rule: if $C \sqcap D \in L(x)$, add both $C$ and $D$ to $L(x)$;
- $\sqcup$-rule: if $C \sqcup D \in L(x)$, *nondeterministically* add $C$ or $D$ — this branching is the source of exponential cost;
- $\exists$-rule: if $\exists R.C \in L(x)$, create a fresh $R$-successor $y$ with $C \in L(y)$;
- $\forall$-rule: if $\forall R.C \in L(x)$ and $x$ has an $R$-successor $y$, add $C$ to $L(y)$.

A branch *clashes* when $\{A, \neg A\} \subseteq L(x)$ and is abandoned. If some branch saturates without a clash, it witnesses a model and $C$ is satisfiable; if every branch clashes, $C$ is unsatisfiable.

Two subtleties are exactly where the easy and hard logics part company:

- **Termination needs blocking.** Transitive roles and $\exists$ can generate infinite successor chains. When a new node's label repeats that of an ancestor, the node is *blocked* and reuses the earlier witness. Inverse roles force the more delicate *pairwise* blocking, since a successor can constrain its predecessor retroactively.
- **Nominals break the tree-model property.** Without $\mathcal{O}$, every logic here enjoys the *tree-model property*: a satisfiable concept has a tree-shaped model, and trees are precisely what blocking tames. A nominal $\{a\}$ must denote one fixed element wherever it occurs,

acting as a global merge point onto which edges fold back, destroying treeness. When $\mathcal{O}$, $\mathcal{I}$ and $\mathcal{Q}$ coincide one gets the notorious *yo-yo* interaction — the technical heart of the $\mathcal{SHOIQ}$ tableau (Horrocks & Sattler, 2005).

## 7 Complexity: the payoff for every letter

The choice of letters is not bookkeeping — it is a choice of *worst-case cost*. For concept satisfiability (with respect to a general TBox):

| Logic | Complexity |
|---|---|
| $\mathcal{ALC}$, $\mathcal{S}$, $\mathcal{SHIN}$ | ExpTime-complete |
| $\mathcal{SHOIN}$, $\mathcal{SHOIQ}$ (OWL 1 DL) | NExpTime-complete |
| $\mathcal{SROIQ}$ (OWL 2 DL) | N2ExpTime-complete |
| $\mathcal{EL}^{++}$ (OWL 2 EL profile) | PTime-complete |

Adding nominals ($\mathcal{O}$) to $\mathcal{SHIN}/\mathcal{SHIQ}$ is precisely the step that lifts satisfiability from ExpTime to NExpTime: nominals add genuine logical power (global identity reasoning), and you pay for it.

Conversely, OWL 2 defines three **profiles** — **EL**, **QL**, **RL** — that deliberately *drop* letters to buy tractability. The logic $\mathcal{EL}^{++}$ behind the **EL** profile discards negation, disjunction, inverse roles and cardinalities, keeping essentially only $\sqcap$ and $\exists$; in return, classification drops to *polynomial* time. This is why SNOMED CT, a medical ontology of some 350,000 classes, is expressed in EL and classifies in practice.

## 8 Where to go next

- **Building or auditing an ontology:** the live question is which profile you can afford, and whether a stray inverse-plus-cardinality axiom is silently costing tractability.
- **Implementing a reasoner:** the work is tableau mechanics, blocking strategies, and the nominal-merging bookkeeping of $\mathcal{SHOIQ}$.
- **Studying the formalism:** complexity frontiers, the tree-model property, and the boundary of decidability.

**Further reading.** Baader, Horrocks, Lutz & Sattler, *An Introduction to Description Logic* (CUP, 2017); the *Description Logic Handbook* (2nd ed., 2007); Horrocks & Sattler, "A Tableau Decision Procedure for $\mathcal{SHOIQ}$" (IJCAI 2005); the W3C *OWL 2 Primer* and *OWL 2 Profiles* recommendations.